# A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. We use the specification of ProVerif's semantics given in Blanchet 16 [1].

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The attacker is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and simple ProVerif queries relating events. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the material formalised in Lean, and of the formal translation from terms in ProVerif's process calculus to our traces. We have not proved the claimed properties of the translation, such as adequacy.

## 1 The trace semantics

### 1.1 Messages

**Definition 1.1** (Signature, terms)**.** A *signature* is a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of sets; $f \in \Sigma_n$ is a constructor of arity $n$. The set $\mathcal{T} = \mathcal{T}(\Sigma)$ of *raw terms* is generated by

$$t ::= \nu_k \ (k \in \mathbb{N}) \ \mid \ f(t_1, \dots, t_n) \ (f \in \Sigma_n).$$

A *substitution* is a map $\sigma : \mathbb{N} \to \mathcal{T}$; $t\sigma$ replaces each $\nu_k$ in $t$ by $\sigma(k)$.

**Definition 1.2** (Theory)**.** A *theory* over $\Sigma$ is a pair $(E, \mathrm{Fr})$ with $E \subseteq \mathcal{T} \times \mathcal{T}$ and $\mathrm{Fr} \subseteq \bigcup_n \Sigma_n$ such that

(H) for every $(l, r) \in E$, both $l$ and $r$ are of the form $f(\dots)$ with $f \notin \mathrm{Fr}$.

The relation $\equiv_E$ is the least congruence on $\mathcal{T}$ containing $l\sigma \equiv_E r\sigma$ for all $(l, r) \in E$ and all substitutions $\sigma$. The set of *messages* is $\mathcal{M} = \mathcal{T}/\!\equiv_E$, with $[t]$ the class of $t$. Constructors act on $\mathcal{M}$ by $f([t_1], \dots, [t_n]) = [f(t_1, \dots, t_n)]$, and $\nu_k$ also denotes $[\nu_k]$.

A $\Sigma$-algebra $A$ *satisfies* $E$ if $l\sigma_A = r\sigma_A$ for every $(l, r) \in E$ and every valuation $\sigma_A : \mathbb{N} \to A$, where $t\sigma_A$ is the evaluation of $t$ in $A$.

**Proposition 1.3.** *$\mathcal{M}$ satisfies $E$. For every $\Sigma$-algebra $A$ satisfying $E$ and every $v : \mathbb{N} \to A$ there is a unique homomorphism $\mathcal{M} \to A$ sending $\nu_k$ to $v(k)$.*

**Definition 1.4** (View)**.** Let $\mathcal{V} = \mathbb{N} \uplus \{f(m_1, \ldots, m_n) \mid f \in \mathrm{Fr} \cap \Sigma_n,\ m_i \in \mathcal{M}\} \uplus \{\bot\}$. Define $\mathrm{view}_0 : \mathcal{T} \to \mathcal{V}$ by

$$\mathrm{view}_0(\nu_k) = k, \qquad \mathrm{view}_0(f(t_1, \ldots, t_n)) = \begin{cases} f([t_1], \ldots, [t_n]) & f \in \mathrm{Fr}, \\ \bot & f \notin \mathrm{Fr}. \end{cases}$$

**Proposition 1.5.** *The following hold.*

1. *$s \equiv_E t$ implies $\mathrm{view}_0(s) = \mathrm{view}_0(t)$; hence $\mathrm{view}_0$ induces $\mathrm{view} : \mathcal{M} \to \mathcal{V}$.*
2. *$\mathrm{view}(f(m_1, \ldots, m_n)) = f(m_1, \ldots, m_n)$ if $f \in \mathrm{Fr}$, and $\bot$ otherwise.*
3. *If $\mathrm{view}(m) = f(m_1, \ldots, m_n)$ then $m = f(m_1, \ldots, m_n)$.*
4. *$k \mapsto \nu_k$ is injective $\mathbb{N} \to \mathcal{M}$.*

**Definition 1.6** (May-fail messages, lifting)**.** Let $\mathcal{M}_\bot = \mathcal{M} \uplus \{\bot\}$. For $g : \mathcal{M}^n \to \mathcal{M}_\bot$ let $g^\uparrow : \mathcal{M}_\bot^n \to \mathcal{M}_\bot$ be $g^\uparrow(\vec{u}) = g(\vec{u})$ if every $u_i \in \mathcal{M}$ and $\bot$ otherwise; $f^\uparrow$ for $f \in \Sigma_n$ is the lift of $f : \mathcal{M}^n \to \mathcal{M} \subseteq \mathcal{M}_\bot$. A *destructor* of arity $n$ is a function $\mathcal{M}_\bot^n \to \mathcal{M}_\bot$.

**Definition 1.7** (Destructors from rewrite rules)**.** Fix disjoint countable sets of *ordinary variables* $x$ and *may-fail variables* $u$. Patterns over Fr are generated by $p ::= x \mid u \mid \mathsf{fail} \mid f(p_1, \ldots, p_n)$ with $f \in \mathrm{Fr} \cap \Sigma_n$. A *matching* of $\vec{u} \in \mathcal{M}_\bot^n$ against $\vec{p}$ is a map $\theta$ from the variables of $\vec{p}$ to $\mathcal{M}_\bot$, sending ordinary variables into $\mathcal{M}$, such that $p_i\theta = u_i$ for all $i$, where $\mathsf{fail}\,\theta = \bot$ and $f(p_1, \ldots, p_n)\theta = f(p_1\theta, \ldots, p_n\theta)$ if every $p_j\theta \in \mathcal{M}$ and is undefined otherwise. By Proposition 1.5, at most one matching exists.

A *right-hand side* is $q ::= \mathsf{fail} \mid u \mid M$, where $M$ is a term over constructors and ordinary variables; $q\theta \in \mathcal{M}_\bot$ is $\bot$, $\theta(u)$, and the evaluation of $M$ in $\mathcal{M}$ under $\theta$ respectively. A list of rules $g(\vec{p}_1) \to q_1;\ \ldots;\ g(\vec{p}_k) \to q_k$, each variable of $q_i$ occurring in $\vec{p}_i$, defines the destructor

$$g(\vec{u}) = \begin{cases} q_j\theta & j \text{ least such that a matching } \theta \text{ of } \vec{u} \text{ against } \vec{p}_j \text{ exists}, \\ \bot & \text{if there is none.} \end{cases}$$

**Definition 1.8** (Public operations, deduction)**.** Let $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and let Dt be a set of destructors. Set $\mathrm{Ops} = \{f^\uparrow \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt}$. A set $S \subseteq \mathcal{M}$ is Ops-*closed* if for every $o \in \mathrm{Ops}$ of arity $n$ and every $\vec{u} \in (S \uplus \{\bot\})^n$, $o(\vec{u}) \in \mathcal{M}$ implies $o(\vec{u}) \in S$. For $K \subseteq \mathcal{M}$,

$$\langle K \rangle = \bigcap \{S \supseteq K \mid S \text{ is Ops-closed}\}, \qquad K \vdash m \iff m \in \langle K \rangle.$$

**Proposition 1.9.** *$\langle\cdot\rangle$ is a closure operator on $\mathcal{P}(\mathcal{M})$, $\langle K \rangle$ is* Ops*-closed, and $K \vdash m_1, \ldots, K \vdash m_n$ imply $K \vdash f(m_1, \ldots, m_n)$ for $f \in \mathrm{Pub} \cap \Sigma_n$. In particular $\emptyset \vdash a$ for $a \in \mathrm{Pub} \cap \Sigma_0$.*

## 1.2 Traces and processes

**Definition 1.10** (Actions, traces)**.**

$$\mathrm{Act} = \{\mathsf{out}(c, m),\ \mathsf{in}(c, m),\ \mathsf{comm}(c, m),\ \mathsf{new}(k),\ \mathsf{ev}(m) \mid c, m \in \mathcal{M},\ k \in \mathbb{N}\}.$$

A *trace* is a word $t \in \mathrm{Act}^*$; $\varepsilon$ is the empty trace and $l \le t$ means $l$ is a prefix of $t$. The actions $\mathsf{out}(c,m)$ and $\mathsf{in}(c,m)$ are *unfinished*; $\mathrm{Closed}(t)$ holds if no action of $t$ is unfinished. $\mathrm{nonces}(t) \in \mathbb{N}^*$ is the image of $t$ under the monoid homomorphism with $\mathsf{new}(k) \mapsto k$ and every other action $\mapsto \varepsilon$. A *process* is a subset of $\mathrm{Act}^*$; $\mathrm{Proc} = \mathcal{P}(\mathrm{Act}^*)$.

**Definition 1.11** (Sequential constructors)**.** For $a \in \mathrm{Act}$, $P \in \mathrm{Proc}$, $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} a \cdot P &= \{\varepsilon\} \cup \{a\,t \mid t \in P\}, & \mathbf{0} &= \{\varepsilon\}, \\ \mathsf{out}(c,m).P &= \mathsf{out}(c,m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c,m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

**Definition 1.12** (Parallel composition)**.** Let $I$ be a set. For $s \in (I \times \mathrm{Act})^*$ and $i \in I$, $s\restriction i \in \mathrm{Act}^*$ is the word of actions of $s$ tagged $i$, in order, and $\overline{s} \in \mathrm{Act}^*$ is $s$ with tags removed. For $f : I \to \mathrm{Act}^*$, $\mathrm{Interleaves}(f,s)$ holds if $s\restriction i = f(i)$ for all $i$. The relation $\mathrm{comms} \subseteq (I \times \mathrm{Act})^* \times \mathrm{Act}^*$ is the least one such that

$$\begin{aligned} &\mathrm{comms}(\varepsilon,\varepsilon), \\ &\mathrm{comms}(s,t) \Rightarrow \mathrm{comms}\big((i,a)\,s,\ a\,t\big), \\ &\mathrm{comms}(s,t) \wedge i \neq j \Rightarrow \mathrm{comms}\big((i,\mathsf{out}(c,m))\,(j,\mathsf{in}(c,m))\,s,\ \mathsf{comm}(c,m)\,t\big). \end{aligned}$$

For $P : I \to \mathrm{Proc}$,

$$\begin{aligned} \mathrm{merge}_I(P) = \{t \mid \exists f, s.\ (\forall i.\ f(i) \in P(i)) \ \wedge\ &\mathrm{Interleaves}(f,s) \\ &\wedge\ \mathrm{comms}(s,t) \ \wedge\ \mathrm{nonces}(t) \text{ has no repeated letter}\}. \end{aligned}$$

$P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$.

**Proposition 1.13.** *Let* $\mathrm{comms}(s,t)$.

1. $\mathrm{nonces}(t) = \mathrm{nonces}(\overline{s})$.
2. *If $a$ occurs in $t$ then $(i,a)$ occurs in $s$ for some $i$, or $a = \mathsf{comm}(c,m)$ and $(i,\mathsf{out}(c,m))$, $(j,\mathsf{in}(c,m))$ occur in $s$ for some $i \neq j$.*
3. *If* $\mathrm{Closed}(t)$ *and $(k,\mathsf{in}(c,m))$ occurs in $s$, then $(i,\mathsf{out}(c,m))$ occurs in $s$ for some $i \neq k$.*
4. *$!P = P \mid !P$ for every $P \in \mathrm{Proc}$.*

## 1.3 Translation of processes

**Definition 1.14** (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail}, \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(N,M); P \mid \mathsf{in}(N,x); P \mid P \mid Q \mid\, !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}(M); P, \end{aligned}$$

where $f$ ranges over constructors and $h$ over constructors and destructors. Types are erased.

**Definition 1.15** (Model)**.** A *model* consists of a signature $\Sigma$ whose nullary symbols include $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$, a theory $(E, \mathrm{Fr})$ over $\Sigma$, a finite set of destructors each given by rules as in Definition 1.7, sets Pub, Dt of public constructors and public destructors, and a closed process $P_0$ with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$.

**Definition 1.16** (Translation)**.** An *environment* $\rho$ maps variables and names to $\mathcal{M}$; $\rho_0$ maps each free name $a$ to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. Expressions are evaluated in $\mathcal{M}_\perp$:

$$[\![M]\!]\rho = \rho M, \qquad [\![\mathsf{fail}]\!]\rho = \perp, \qquad [\![f(\vec{D})]\!]\rho = f^{\uparrow}([\![\vec{D}]\!]\rho), \qquad [\![g(\vec{D})]\!]\rho = g([\![\vec{D}]\!]\rho)$$

for a constructor $f$ and a destructor $g$. Processes are translated by

$$\begin{aligned}
[\![\mathbf{0}]\!]\rho &= \mathbf{0}, & [\![\mathsf{out}(N,M);P]\!]\rho &= \mathsf{out}(\rho N, \rho M).[\![P]\!]\rho,\\
[\![\mathsf{event}(M);P]\!]\rho &= \mathsf{ev}(\rho M).[\![P]\!]\rho, & [\![\mathsf{in}(N,x);P]\!]\rho &= \mathsf{in}(\rho N).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big),\\
[\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho, & [\![\mathsf{new}\ a;P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big),\\
[\![!P]\!]\rho &= ![\![P]\!]\rho, & [\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho[x \mapsto m] & [\![D]\!]\rho = m \in \mathcal{M},\\ [\![Q]\!]\rho & [\![D]\!]\rho = \perp, \end{cases}\\
& & [\![\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho &= \begin{cases} [\![P]\!]\rho & \rho M = \mathsf{true},\\ [\![Q]\!]\rho & \rho M \neq \mathsf{true}. \end{cases}
\end{aligned}$$

The translation of $P_0$ is $[\![P_0]\!]\rho_0$. A translated process mentions no $\nu_k$ except through $\mathsf{new}$.

*Remark* 1.17. In the development a process is a program in the continuation monad with answer type Proc, run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c,m) = \lambda\kappa.\,\mathsf{out}(c,m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation.

## 1.4 The attacker and the security properties

**Definition 1.18** (Enemy)**.** The processes $\mathrm{Enemy}(K) \in \mathrm{Proc}$, for $K \subseteq \mathcal{M}$, are the pointwise least such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c,m)\,e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;
3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c,m)\,e \in \mathrm{Enemy}(K)$;
4. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{comm}(c,m)\,e \in \mathrm{Enemy}(K)$, an output to itself;
5. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\,e \in \mathrm{Enemy}(K)$.

**Definition 1.19** (Attacks, knowledge)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset)\ :\ \mathrm{Closed}(t)\}.$$

For $K \subseteq \mathcal{M}$, $\mathrm{known}(K,\cdot) : \mathrm{Act}^* \to \mathcal{P}(\mathcal{M})$ is defined by

$$\begin{aligned}
\mathrm{known}(K,\varepsilon) &= K,\\
\mathrm{known}(K,a\,t) &= \mathrm{known}\big(\langle K \cup \{m \mid a = \mathsf{comm}(c,m),\ c \in K\}\rangle,\ t\big).
\end{aligned}$$

**Proposition 1.20** (Prefix-closure)**.** Enemy($K$) *and every process built from the constructors of Definitions 1.11 and 1.12 are prefix-closed. If $P$ is prefix-closed, so is* Attacks($P$)*.*

**Definition 1.21** (Facts, secrecy, correspondence)**.** A *fact* is $\mathsf{event}(e)$ or $\mathsf{knows}(m)$ with $e, m \in \mathcal{M}$; Fact is the set of facts. A fact *holds* after $t \in \mathrm{Act}^*$, written $t \models f$, if

$$t \models \mathsf{event}(e) \iff \mathsf{ev}(e) \in t, \qquad t \models \mathsf{knows}(m) \iff m \in \mathrm{known}(\langle\emptyset\rangle, t).$$

For a set $X$, $F, H : (X \to \mathcal{M}) \to \mathrm{Fact}$ and $s \in \mathcal{M}$,

$$\begin{aligned}\mathrm{Corr}(P, F, H) \iff{}& \forall \sigma : X \to \mathcal{M}.\ \forall t \in \mathrm{Attacks}(P).\\ & t \models F(\sigma) \Rightarrow \exists \sigma' : X \to \mathcal{M}.\ F(\sigma') = F(\sigma) \wedge t \models H(\sigma'),\\ \mathrm{Secret}(P, s) \iff{}& \forall t \in \mathrm{Attacks}(P).\ t \not\models \mathsf{knows}(s).\end{aligned}$$

By Proposition 1.20, for prefix-closed $P$ these are properties of every prefix of every attack: $\mathrm{Corr}(P, F, H)$ says that some $H(\sigma')$ holds by the time $F(\sigma)$ does.

**Definition 1.22** (Queries)**.** A *query fact* is $\mathsf{event}(M)$ or $\mathsf{attacker}(M)$ with $M$ a term over constructors, free names and a finite set $X$ of variables; it is an *event fact* or an *attacker fact* accordingly. For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of $M$ with free names as constants, and let the *instance* of a query fact under $\sigma$ be $\sigma\,\mathsf{event}(M) = \mathsf{event}(\sigma M)$ and $\sigma\,\mathsf{attacker}(M) = \mathsf{knows}(\sigma M)$. The query $\mathsf{query\ attacker}(M)$, for $M$ without variables, denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$. The query $\mathsf{query}\ F \Longrightarrow H$, with $F$ and $H$ query facts over $X$, denotes $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$, the survey's Definition 3.5 with $=$ read as $=_E$.

# 2 Relation to the operational semantics of ProVerif

## 2.1 The reduction semantics

This section restates the semantics of Blanchet's survey (§2.4, §2.5.1, §3.1–3.3) for the syntax of Definition 1.14. Names are atoms; substitutions map variables to terms and never substitute $\mathsf{fail}$ or a may-fail variable for an ordinary variable. An equational theory $E$ is a finite set of equations between terms without names; $=_E$ is the closure of $E$ under reflexivity, symmetry, transitivity, application of function symbols and substitution. Each constructor $f$ carries the rule $f(\vec{x}) \to f(\vec{x})$, each destructor $g$ a finite ordered list of rules $g(\vec{U}) \to U$ between may-fail terms without names, and every $\mathrm{def}(h)$ ends with $h(\vec{u}) \to \mathsf{fail}$ for may-fail variables $\vec{u}$. The survey states §3 for the core calculus; here its definitions are taken modulo $=_E$: a step on a term $M' =_E M$ counts as a step on $M$, and substitutions are compared modulo $=_E$.

Evaluation $D \Downarrow U$ of closed expressions to closed may-fail terms is the least relation with $M \Downarrow M$, $\mathsf{fail} \Downarrow \mathsf{fail}$, and $h(D_1, \ldots, D_n) \Downarrow \sigma U'_j$ whenever $D_i \Downarrow U_i$ for all $i$, $\mathrm{def}(h) = (h(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \le k}$, $\sigma U'_{j,i} =_E U_i$ for all $i$, and for all $i < j$ and all $\sigma'$ there is some $l$ with $\sigma' U'_{i,l} \neq_E U_l$.

A *configuration* is $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P}$ with $\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}$ finite disjoint sets of names and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Writing $\mathcal{E}$ for the pair of name sets,

reduction is the least relation closed under

$$\mathcal{E}, \mathcal{P} \cup \{\mathbf{0}\} \to \mathcal{E}, \mathcal{P} \qquad \text{(Red Nil)}$$

$$\mathcal{E}, \mathcal{P} \cup \{P \mid Q\} \to \mathcal{E}, \mathcal{P} \cup \{P, Q\} \qquad \text{(Red Par)}$$

$$\mathcal{E}, \mathcal{P} \cup \{!P\} \to \mathcal{E}, \mathcal{P} \cup \{P, !P\} \qquad \text{(Red Repl)}$$

$$(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \mathcal{P} \cup \{\mathsf{new}\ a; P\} \to (\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}} \cup \{a'\}), \mathcal{P} \cup \{P\{a'/a\}\} \quad \text{if } a' \notin \mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}} \qquad \text{(Red Res)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} \to \mathcal{E}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} \quad \text{if } N =_E N' \qquad \text{(Red I/O}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\{M/x\}\} \quad \text{if } D \Downarrow M \qquad \text{(Red Eval 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Red Eval 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \quad \text{if } M =_E \mathsf{true} \qquad \text{(Red Cond 1}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } M \neq_E \mathsf{true} \qquad \text{(Red Cond 2}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{event}(M); P\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \qquad \text{(Red Event)}$$

A *trace* is a finite reduction sequence $\mathcal{E}_0, \mathcal{P}_0 \to \cdots \to \mathcal{E}_n, \mathcal{P}_n$, written $\mathcal{E}_0, \mathcal{P}_0 \to^* \mathcal{E}_n, \mathcal{P}_n$.

**Definition 2.1** (Survey, Definitions 3.1–3.5)**.** Let $P_0$ be a closed process.

1. A closed process $Q$ is an $\mathcal{N}_{\text{pub}}$-*adversary* if $\text{fn}(Q) \subseteq \mathcal{N}_{\text{pub}}$, every function symbol of $Q$ is public, and $Q$ contains no $\mathsf{event}$.

2. A trace *outputs $M$ publicly* if it contains a step (Red I/O′) whose output is $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\text{pub}}$ and $M' =_E M$. It *executes* $\mathsf{event}(M)$ if it contains a step (Red Event) on $\mathsf{event}(M'); P$ with $M' =_E M$.

3. $P_0$ *preserves the secrecy of* the closed term $M$ *from* $\mathcal{N}_{\text{pub}}$ if, for some $\mathcal{N}_{\text{priv}}$ disjoint from $\mathcal{N}_{\text{pub}}$ with $\text{fn}(M) \cup \text{fn}(P_0) \subseteq \mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}}$, no trace $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\} \to^* \mathcal{E}', \mathcal{P}'$ with $Q$ an $\mathcal{N}_{\text{pub}}$-adversary outputs $M$ publicly.

4. $P_0$ *satisfies* $\mathsf{event}(M) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} \mathsf{event}(M_{jk})$ *against* $\mathcal{N}_{\text{pub}}$*-adversaries* if, for some $\mathcal{N}_{\text{priv}}$ as above, for every $\mathcal{N}_{\text{pub}}$-adversary $Q$, every trace Tr from $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\}$ and every substitution $\sigma$: if Tr executes $\mathsf{event}(\sigma M)$ then there are $\sigma'$ and $j$ with $\sigma' M =_E \sigma M$ such that Tr executes $\mathsf{event}(\sigma' M_{jk})$ for every $k \leq l_j$.

Fix $\mathcal{N}_{\text{priv}}$ as in Definition 2.1, and for an $\mathcal{N}_{\text{pub}}$-adversary $Q$ write $\mathcal{C}_Q$ for the configuration $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\}$.

**Definition 2.2** (Satisfaction)**.** A trace *satisfies* $\mathsf{attacker}(M)$ if it outputs $M$ publicly, and *satisfies* $\mathsf{event}(M)$ if it executes $\mathsf{event}(M)$.

**Definition 2.3** (Correspondences between facts)**.** Let $F$ and $H_{jk}$ ($j \leq m$, $k \leq l_j$) be query facts over $X$. $P_0$ *satisfies* $F \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} H_{jk}$ *against* $\mathcal{N}_{\text{pub}}$*-adversaries* if for every $\mathcal{N}_{\text{pub}}$-adversary $Q$, every trace Tr from $\mathcal{C}_Q$ and every substitution $\sigma$: if Tr satisfies $\sigma F$ then there are $\sigma'$ and $j$ with $\sigma' F =_E \sigma F$ such that Tr satisfies $\sigma' H_{jk}$ for every $k \leq l_j$. For event facts this is Definition 2.1(4).

## 2.2 The correspondence

Throughout, fix a model (Definition 1.15) satisfying

(R1) Fr is the set of symbols at the root of no equation and $(E, \mathrm{Fr})$ satisfies (H);

(R2) every rule of every destructor has patterns over Fr, and the destructor is the one of Definition 1.7;

(R3) Pub consists of the public constructors and the names in $\mathcal{N}_{\mathrm{pub}}$, and Dt of the public destructors;

(R4) $P_0$ is a closed process of Definition 1.14.

These conditions exclude some ProVerif models: an equation with a variable as one side, such as the survey's (2.8), violates (H), and a destructor whose rules match on a symbol at the root of an equation has no counterpart under (R2).

Names of the reduction semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection $\iota$ between the names outside $\mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, those that (Red Res) may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$ and $\iota\,\mathsf{fail} = \bot$; $\iota M$ depends only on the class of $M$ modulo $E$. Let $\rho_\iota$ be the environment with $\rho_\iota(a) = \iota a$ for every free or created name $a$.

**Lemma 2.4** (Evaluation). *For every closed expression $D$ over free and created names, some $U$ satisfies $D \Downarrow U$, and $D \Downarrow U$ implies $\iota U = [\![D]\!]\rho_\iota$.*

**Definition 2.5** (Labelled semantics). A *labelled configuration* is $(U, \mathcal{P})$ with $U \subseteq \mathbb{N}$ finite and $\mathcal{P}$ a finite multiset of closed processes whose names are free names and symbols $\nu_k$. The $\tau$-steps $(U, \mathcal{P}) \xrightarrow{\tau} (U, \mathcal{P}')$ are the steps (Red Nil), (Red Par), (Red Repl), (Red Eval 1), (Red Eval 2), (Red Cond 1′) and (Red Cond 2′), with $=_E$ read on $\mathcal{M}$. The visible steps are

$$
\begin{aligned}
(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) &\xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),\\
(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) &\xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M,\\
(U, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\}) &\xrightarrow{\mathsf{comm}([N],[M])} (U, \mathcal{P} \cup \{Q,\ P\{M/x\}\}) \quad \text{if } N =_E N',\\
(U, \mathcal{P} \cup \{\mathsf{new}\ a; P\}) &\xrightarrow{\mathsf{new}(k)} (U \cup \{k\}, \mathcal{P} \cup \{P\{\nu_k/a\}\}) \quad \text{for every } k \notin U,\\
(U, \mathcal{P} \cup \{\mathsf{event}(M); P\}) &\xrightarrow{\mathsf{ev}([M])} (U, \mathcal{P} \cup \{P\}).
\end{aligned}
$$

The third step is (Red I/O′) between two members of $\mathcal{P}$. It is visible rather than a $\tau$-step because a trace records a completed communication, which is what known reads the attacker's knowledge off. With $\overset{\alpha}{\Longrightarrow} = \xrightarrow{\tau}{}^{*} \xrightarrow{\alpha} \xrightarrow{\tau}{}^{*}$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Act}^* \mid (U, \mathcal{P}) \overset{\alpha_1}{\Longrightarrow} \cdots \overset{\alpha_n}{\Longrightarrow}\}.$$

**Theorem 2.6** (Adequacy). *For every closed process $P$ whose names are free names, $[\![P]\!]\rho_0 = \mathrm{Tr}(\emptyset, \{P\})$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes, $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.*

**Definition 2.7** (Observation)**.** For a trace Tr from $\mathcal{C}_Q$, $\mathrm{obs}(\mathrm{Tr}) \in \mathrm{Act}^*$ is the word that records $\mathsf{new}(\iota a')$ for each step (Red Res) creating $a'$ and $\mathsf{ev}(\iota M)$ for each step (Red Event) on $\mathsf{event}(M)$, in the order of the steps, and nothing for the other steps. For $t \in \mathrm{Act}^*$, $t{\restriction_{\mathsf{new,ev}}}$ is the subword of new and ev actions.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$.

**Theorem 2.8** (Attacks)**.** *The following hold.*

*(S) For every $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$ and every trace* Tr *from $\mathcal{C}_Q$ there is $t \in \mathcal{A}_0$ with $t{\restriction_{\mathsf{new,ev}}} = \mathrm{obs}(\mathrm{Tr})$ such that $\iota M \in \mathrm{known}(\langle\emptyset\rangle, t)$ whenever* Tr *outputs $M$ publicly.*

*(C) For every $t \in \mathcal{A}_0$ there are an $\mathcal{N}_{\mathrm{pub}}$-adversary $Q$ and a trace* Tr *from $\mathcal{C}_Q$ with $\mathrm{obs}(\mathrm{Tr}) = t{\restriction_{\mathsf{new,ev}}}$. If moreover $m \in \mathrm{known}(\langle\emptyset\rangle, t)$ and $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, they can be chosen so that the last step of* Tr *outputs publicly a term $M$ with $\iota M = m$.*

**Theorem 2.9** (Secrecy)**.** *Let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, let $M$ be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ and let $s = \rho_0 M$. Then $\mathrm{Secret}([\![P_0]\!]\rho_0, s)$ holds if and only if $P_0$ preserves the secrecy of $M$ from $\mathcal{N}_{\mathrm{pub}}$.*

*Remark* 2.10. The hypothesis $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ cannot be dropped: when $\mathcal{N}_{\mathrm{pub}} = \emptyset$ no trace outputs anything publicly, so $P_0$ preserves the secrecy of every $M$, whereas $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$ fails whenever $\emptyset \vdash \rho_0 M$, for instance for a public constant $M$.

**Theorem 2.11** (Correspondences)**.** *Let $F$ and $H$ be query facts over $X$ such that $H$ is an event fact, and let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ if $F$ is an attacker fact. Then $P_0$ satisfies $F \rightsquigarrow H$ against $\mathcal{N}_{\mathrm{pub}}$-adversaries if and only if $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$.*

*Remark* 2.12. Neither hypothesis can be dropped. When $\mathcal{N}_{\mathrm{pub}} = \emptyset$ and $F = \mathsf{attacker}(M)$, $P_0$ satisfies the correspondence vacuously, whereas Corr fails as soon as $\emptyset \vdash \sigma M$ for some valuation $\sigma$. For $H = \mathsf{attacker}(M)$, Definition 2.2 requires $\sigma M$ itself to be output publicly, whereas $\mathsf{knows}(\sigma M)$ holds as soon as $\sigma M$ is derivable: with the symmetric encryption of the survey's §2.1, $\mathcal{N}_{\mathrm{pub}} = \{c\}$ and $P_0 = \mathsf{out}(c, \mathsf{senc}(s, k)); \mathsf{out}(c, k); \mathsf{event}(e)$, Corr holds for $\mathsf{event}(e) \rightsquigarrow \mathsf{attacker}(s)$ while the correspondence of Definition 2.3 fails. The reading by derivability is ProVerif's.

# References


[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.